PRACTICAL - 03

Aim: Distinct points (Point.java + Driver.java)

Code Screenshot:

File Edit Selection View Go Run Terminal Help

Point.java

Point.java > ...

```java
import java.util.Objects;

public class Point {
    private int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

Java

File Edit Selection View Go Run Terminal Help

Driver.java

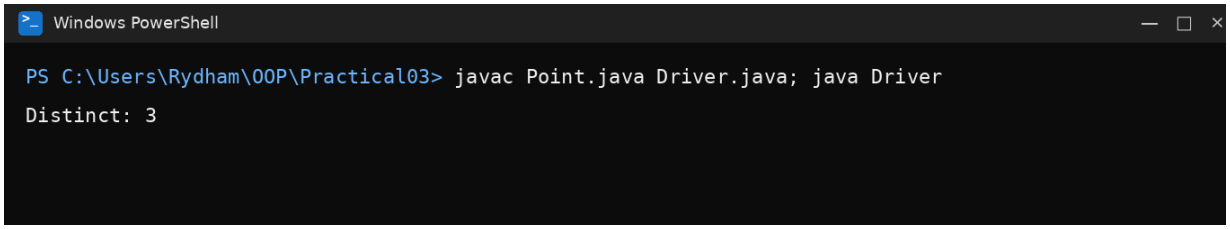
Driver.java > ...

```java
public class Driver {
    public static void main(String[] args) {
        Point[] points = {
            new Point(1, 2), new Point(3, 4),
            new Point(1, 2), new Point(5, 6),
            new Point(3, 4)
        };

        int distinct = 0;
        for (int i = 0; i < points.length; i++) {
            boolean found = false;
            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) {
                    found = true;
                    break;
                }
            }
            if (!found) distinct++;
        }
        System.out.println("Distinct: " + distinct);
    }
}
```

Java

Output Screenshot:



Learning Outcome:

- Mastered the contract between equals() and hashCode() in Java.
- Overrode Object.toString() for human-readable coordinate representation.
- Implemented array deduplication logic using equality checking.
- Learned pattern matching for instanceof introduced in modern Java.
- Understood hashing consistency requirements when overriding equals.

PRACTICAL - 03

Aim: Duplicate card check (Card.java + Driver.java)

Code Screenshot:

File Edit Selection View Go Run Terminal Help

Card.java

Card.java > ...

```java
import java.util.Objects;

public class Card {
    private String rank, suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String toString() {
        return rank + " of " + suit;
    }

    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Card c)) return false;
        return Objects.equals(rank, c.rank)
            && Objects.equals(suit, c.suit);
    }

    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

Java

File Edit Selection View Go Run Terminal Help

Driver.java

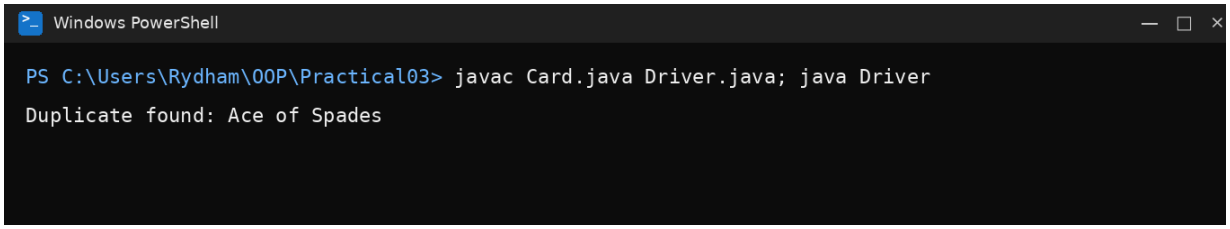
Driver.java > ...

```java
public class Driver {
    public static void main(String[] args) {
        Card[] input = {
            new Card("Ace", "Spades"),
            new Card("King", "Hearts"),
            new Card("Ace", "Spades")
        };
        Card[] cards = new Card[input.length];
        int count = 0;

        for (Card card : input) {
            boolean duplicate = false;
            for (int i = 0; i < count; i++) {
                if (card.equals(cards[i])) {
                    System.out.println("Duplicate found: " + card);
                    duplicate = true;
                    break;
                }
            }
            if (duplicate) break;
            cards[count++] = card;
        }
    }
}
```

Java

Output Screenshot:



Learning Outcome:

- Modeled compound domain entities with composite equality semantics.
- Applied string formatting in overridden toString() representations.
- Detected duplicates with early-exit search algorithms.
- Understood how Objects.equals() safely handles null references.
- Reinforced object identity vs structural equality concepts.

PRACTICAL - 03

Aim: Fractions (Fraction.java + Driver.java) - Advanced Learners

Code Screenshot:

File Edit Selection View Go Run Terminal Help

Fraction.java

```java
import java.util.Objects;

public class Fraction {
    private int num, den;

    public Fraction(int num, int den) {
        if (den == 0)
            throw new IllegalArgumentException("Denominator cannot be zero");
        int g = gcd(Math.abs(num), Math.abs(den));
        this.num = num / g;
        this.den = den / g;
        if (this.den < 0) {
            this.num = -this.num;
            this.den = -this.den;
        }
    }

    private int gcd(int a, int b) {
        while (b != 0) {
            int t = b;
            b = a % b;
            a = t;
        }
        return a;
    }

    public String toString() {
        return num + "/" + den;
    }

    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Fraction f)) return false;
        return num == f.num && den == f.den;
    }

    public int hashCode() {
        return Objects.hash(num, den);
    }
}
```

Java

File Edit Selection View Go Run Terminal Help

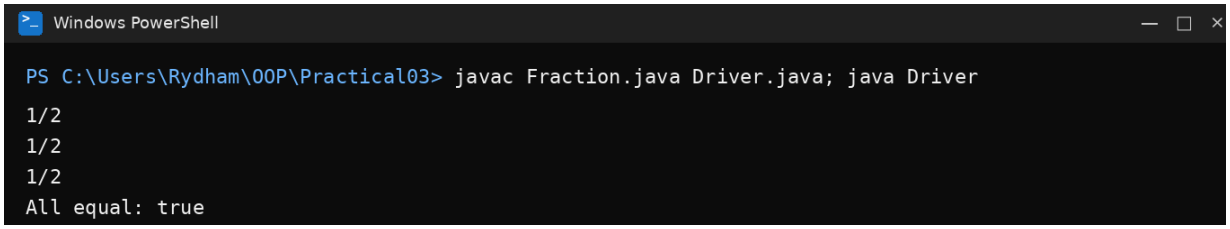
Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Fraction f1 = new Fraction(1, 2);
        Fraction f2 = new Fraction(2, 4);
        Fraction f3 = new Fraction(3, 6);

        System.out.println(f1);
        System.out.println(f2);
        System.out.println(f3);
        System.out.println("All equal: " +
            (f1.equals(f2) && f2.equals(f3)));
    }
}
```

Java

Output Screenshot:



Learning Outcome:

- Reduced fractions to their lowest terms using the GCD method.
- Overrode toString() to display fractions in readable form.
- Compared equivalent fractions using equals().
- Kept hashCode() consistent with the equality comparison.
- Represented rational values using a Java class.